

Experiment No. 01:

Experiment Title: Controlling the Light Emitting Diode (LED) with a push button.

Objectives

- Write an embedded C/C++ program in Arduino IDE to read digital input and produce digital output.
- Interface an LED and a tactile push button with an Arduino Uno using breadboard connections.
- Understand the role and necessity of current-limiting resistors and pull-down resistors in digital circuits.
- Observe real-time digital pin state transitions through physical/simulation hardware.

Introduction

The Arduino Uno is a widely used open-source microcontroller board based on the ATmega328P processor. It serves as the main brain in prototyping simple to complex electronic projects.

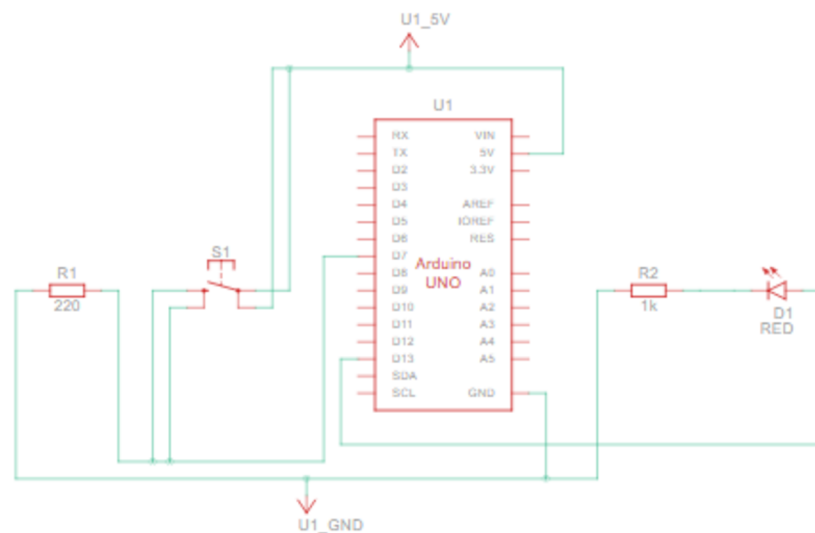
Pin Configuration & Features:

- Digital I/O Pins: 14 digital input/output pins (Pins 0–13), of which 6 can supply PWM output.
- Analog Input Pins: 6 analog inputs (A0–A5) for measuring variable voltages (0V–5V).
- Power Pins: Supplies 5V, 3.3V, and GND connections to power external modules.
- Operating Voltage: Operates at 5V, powered via a USB Type-B cable or external DC jack (7V–12V).
- Clock Speed: Runs at a crystal oscillator speed of 16 MHz.

Apparatus Required

Component / Equipment	Quantity	Specification / Description
Arduino Uno R3	1	Microcontroller development board
LED	1	Standard 5mm Light Emitting Diode
Push Button	1	Momentary tactile switch
Resistor (220 Ω)	1	Current-limiting resistor for the LED
Resistor (10k Ω)	1	Pull-down resistor for the push button
Breadboard	1	Solderless prototyping board
Jumper Wires	As needed	Male-to-Male jumper wires

Circuit Diagram



Working Procedure

1. Place the Arduino Uno, LED, Push Button, and resistors onto the Tinkercad workspace.
2. Connect the cathode (short leg) of the LED directly to the GND pin of the Arduino.
3. Connect the anode (long leg) of the LED to a 220 Ω resistor, and connect the other end of the resistor to Digital Pin 13 of the Arduino.
4. Place the push button across the center divider of the breadboard.
5. Connect one terminal of the push button to the 5V pin on the Arduino.
6. Connect the second terminal of the push button to Digital Pin 2 of the Arduino.
7. Connect a 10k Ω pull-down resistor from the same second terminal of the push button to GND to prevent a floating input state.
8. Open the code editor in Tinkercad, paste the program, compile, and click Start Simulation.
9. Press the push button to observe the LED turning ON, and release it to see the LED turn OFF.

Code

```
// C++ code

int ledpin =13;

int inpin =7;

int val =5;

void setup(){

  pinMode(ledpin, OUTPUT);

  pinMode(inpin, INPUT);}

void loop(){ val=digitalRead(inpin);

  if(val==HIGH{digitalWrite(ledpin,HIGH); }

  else{digitalWrite(ledpin,LOW); }}
```

Discussion and Conclusion

- Discussion: The circuit operated as expected. Using a $10\text{k}\Omega$ pull-down resistor kept Digital Pin 2 anchored at 0V (LOW) when unpressed, preventing floating noise from inadvertently triggering the LED. Pressing the switch connected 5V directly to Pin 2, generating a stable HIGH signal. The 220Ω resistor effectively protected the LED from excessive current.
- Final Verdict: The experiment was completed successfully. The microcontroller accurately detected the state change of the push button input and toggled the LED output accordingly.